Now, if we configure our login and run our sleeper function, we can see that everything works just fine (Figure 3). We can clearly see these lines of code. If we start to use this function in our code, we can see that neither do we have the signature highlighted any longer nor are the arguments autocompleted. If we pass an argument that shouldn't be there, it is not highlighted and IDE does not indicate that we are doing something wrong.

"What are the reasons for this?" you may ask. Technically, it seems perfectly fine behaviour because decorators are substituting one function for another. Here, we defined a function called 'wrapper' and substituted our original function with the *wrapper* function. Upon importing the inspect module, when we used the *inspect.signature* function to just view the signature, we could see that the signature is still there (Figure 4).

The trick is that *functools.wrap* actually preserves the signature in a specific attribute and it is there at runtime. And that begs the question: "Why could our IDE not pick the information up?"

Static code analysis

The reason for this is exactly what has been mentioned above. The information about the original signature is stored in an attribute and is available at runtime. But IDE does not run our code. Instead, it uses static code analysis; so it does not have access to runtime objects and there are plenty of reasons for that.

- *Side effects:* If you are running the code without proper knowledge of it, you may end up damaging your hard drive.
- *Code may raise an exception or run for an infinite time:* There could also be an exception in the code that can be raised while it's being run.
- *Syntactically incorrect code:* The code may not be syntactically correct code.

In static code analysis, IDEs understand the code itself without actually running it, which is a much more difficult task to accomplish.

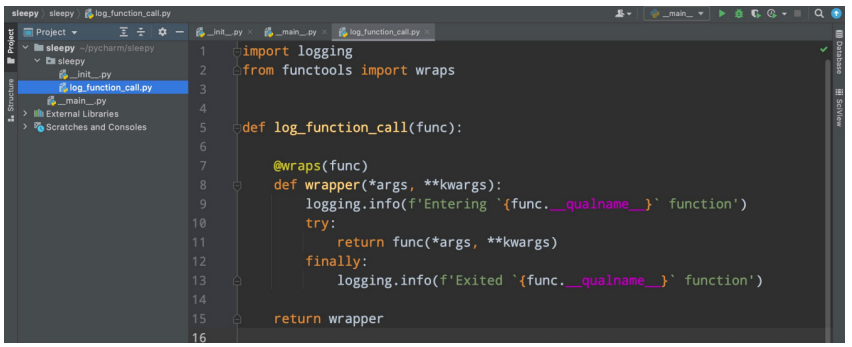

Figure 2: A simple decorator

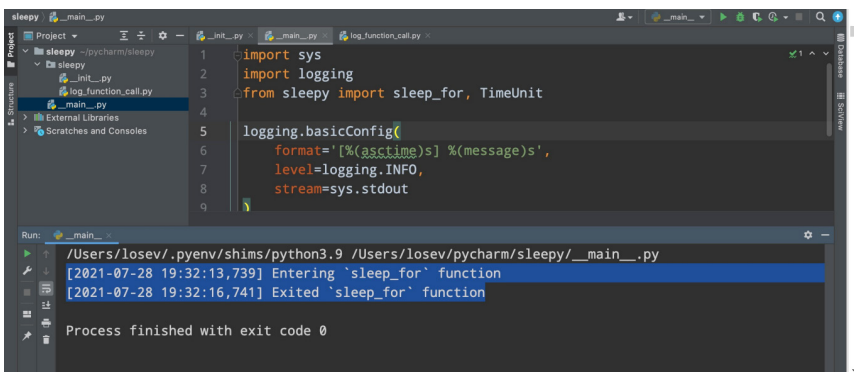

Figure 3: Function output

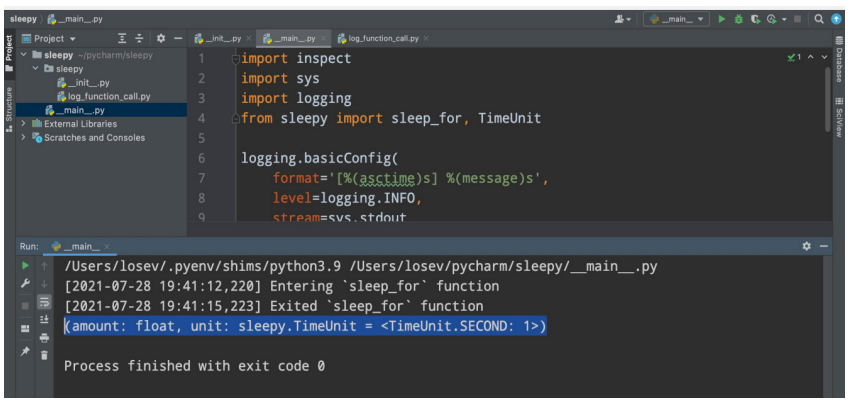

Figure 4: Screen with signature

So what can we do?

Generally, it is the best practice to provide the IDE with as much information as possible in the code itself. But in this particular case, this is what can be done. Figure 5 shows what our original decorator looked like; we can provide additional information in the form of typing. We can tell our IDE that the log function call is accepting and returning something of the same type. In our case, if the *log_function_call* accepts a function with some signature, then it should return the function with the same signature.

Let us see how this works. If we start typing the argument name, we can see that there is a pop-up window above, which lists the whole signature (Figure 5). So actually the information